

## Réalisation d'un service numérique de données territoriales

## Épreuve E4 – Certification Data Engineer

**EL BREK HAMZA**

21 janvier 2026

**Table des matières**

|                                                                    |           |
|--------------------------------------------------------------------|-----------|
| <b>Introduction générale</b>                                       | <b>3</b>  |
| <b>1 Architecture globale du service numérique</b>                 | <b>3</b>  |
| 1.1 Architecture du dépôt GitHub . . . . .                         | 3         |
| 1.2 Vue d'ensemble de l'architecture . . . . .                     | 3         |
| 1.3 Choix technologiques . . . . .                                 | 4         |
| 1.4 Description des composants du système . . . . .                | 4         |
| <b>2 C8 – Automatisation de l'extraction des données</b>           | <b>4</b>  |
| 2.1 Identification et description des sources de données . . . . . | 4         |
| 2.2 Spécifications techniques de l'extraction . . . . .            | 8         |
| 2.3 Scripts d'extraction des données . . . . .                     | 9         |
| 2.4 Automatisation et pérennisation de la collecte . . . . .       | 9         |
| 2.5 Gestion des erreurs et des exceptions . . . . .                | 10        |
| 2.6 Versionnement et accès au dépôt Git . . . . .                  | 10        |
| <b>3 C9 – Requêtes SQL pour la collecte des données</b>            | <b>10</b> |
| 3.1 Présentation du système interrogé . . . . .                    | 10        |
| 3.2 Requêtes SQL de collecte . . . . .                             | 10        |
| 3.3 Choix de sélection, filtrage et jointures . . . . .            | 11        |
| 3.4 Optimisation des requêtes SQL . . . . .                        | 11        |
| 3.5 Documentation des requêtes . . . . .                           | 11        |
| <b>4 C10 – Agrégation, nettoyage et normalisation des données</b>  | <b>12</b> |
| 4.1 Présentation du processus d'agrégation . . . . .               | 12        |
| 4.2 Script d'agrégation des données . . . . .                      | 12        |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 4.3      | Nettoyage des données . . . . .                                            | 12        |
| 4.4      | Homogénéisation et normalisation . . . . .                                 | 13        |
| 4.5      | Documentation et exécution . . . . .                                       | 13        |
| 4.6      | Conformité RGPD . . . . .                                                  | 13        |
| <b>5</b> | <b>C11 – Création et alimentation de la base de données</b>                | <b>13</b> |
| 5.1      | Modélisation MERISE . . . . .                                              | 13        |
| 5.1.1    | Modèle Conceptuel de Données (MCD) . . . . .                               | 13        |
| 5.1.2    | Modèle Logique de Données (MLD) . . . . .                                  | 14        |
| 5.1.3    | Modèle Physique de Données (MPD) . . . . .                                 | 15        |
| 5.2      | Installation et configuration . . . . .                                    | 16        |
| 5.3      | Script d'import des données . . . . .                                      | 17        |
| <b>6</b> | <b>C12 – Mise à disposition via API REST</b>                               | <b>18</b> |
| 6.1      | Présentation de l'API . . . . .                                            | 18        |
| 6.2      | Description des endpoints . . . . .                                        | 19        |
| 6.3      | Règles d'authentification . . . . .                                        | 19        |
| 6.4      | Documentation OpenAPI . . . . .                                            | 19        |
| 6.5      | Démonstration Postman . . . . .                                            | 20        |
| <b>7</b> | <b>Analyse des difficultés et vigilances</b>                               | <b>22</b> |
| 7.1      | Difficultés techniques rencontrées . . . . .                               | 22        |
|          | <b>Conclusion</b>                                                          | <b>22</b> |
| <b>A</b> | <b>Modélisation de la base de données</b>                                  | <b>23</b> |
| A.1      | Schéma MCD . . . . .                                                       | 23        |
| A.2      | Script SQL complet de création . . . . .                                   | 23        |
| <b>B</b> | <b>Scripts Python complets</b>                                             | <b>24</b> |
| B.1      | Script d'extraction des communes (fetch_communes.py) . . . . .             | 24        |
| B.2      | Script de préparation des données (data_prep.py) . . . . .                 | 26        |
| B.3      | Script d'export SQL (export_to_sql.py) . . . . .                           | 29        |
| B.4      | Notebook Databricks – parquet_etl (PySpark) . . . . .                      | 32        |
| B.5      | Script export Azure SQL projet_data_eng_bis (sql_to_adls_bis.py) . . . . . | 35        |
| <b>C</b> | <b>Tests API avec Postman</b>                                              | <b>36</b> |
| C.1      | Test endpoint /health . . . . .                                            | 37        |
| C.2      | Test endpoint /communes avec authentification . . . . .                    | 38        |
| C.3      | Test rejet sans API Key . . . . .                                          | 38        |
| <b>D</b> | <b>Références</b>                                                          | <b>38</b> |

# Introduction générale

Dans un contexte de transformation numérique accélérée, les établissements bancaires s'appuient sur la donnée pour piloter leurs décisions stratégiques. L'exploitation de données fiables, structurées et à jour constitue un levier essentiel pour le développement de nouveaux services et la croissance à long terme.

L'entreprise s'inscrit dans un plan stratégique à horizon 2030, dont l'expansion territoriale constitue l'un des axes majeurs. Cette orientation nécessite une connaissance approfondie des territoires ciblés à travers l'analyse de leurs caractéristiques socio-économiques, démographiques et géographiques.

Le présent projet vise à concevoir un service numérique permettant la collecte, le traitement, le stockage et la mise à disposition de données territoriales issues de sources hétérogènes. Ces données alimentent des études destinées à accompagner la prise de décision des équipes métiers et techniques.

Ce projet mobilise cinq compétences clés en data engineering :

- **C8** : Automatisation de l'extraction depuis sources multiples
- **C9** : Développement de requêtes SQL optimisées
- **C10** : Agrégation et normalisation des données
- **C11** : Création et modélisation de base de données
- **C12** : Exposition des données via API REST

Il a été développé dans le respect des contraintes techniques et réglementaires, notamment la conformité RGPD.

## 1 Architecture globale du service numérique

### 1.1 Architecture du dépôt GitHub

Le code source est hébergé sur GitHub : <https://github.com/haelbrek/Projet-Data-ENG>.

**Structure organisée en couches fonctionnelles :**

- **ingestion/** : Scripts de collecte depuis API REST et scraping
- **analytics/** : Scripts de transformation et export SQL
- **data/** : Stockage local temporaire des données collectées
- **docs/** : Documentation technique et procédures

### 1.2 Vue d'ensemble de l'architecture

L'architecture suit un pipeline classique ETL (Extract, Transform, Load) :

- **Couche 1 – Extraction** : Collecte automatisée depuis sources hétérogènes (API Géo, CSV institutionnels, scraping Meilleurtaux).
- **Couche 2 – Transformation** : Nettoyage, agrégation et normalisation via Python/Pandas dans des scripts dédiés.
- **Couche 3 – Stockage** : Modélisation relationnelle et import dans Azure SQL Database.
- **Couche 4 – Exposition** : API REST FastAPI pour mise à disposition sécurisée aux applications consommatrices.

### 1.3 Choix technologiques

| Composant                 | Technologie        | Justification                       |
|---------------------------|--------------------|-------------------------------------|
| Base de données           | Azure SQL Database | SGBDR robuste, sauvegardes auto     |
| Extraction/Transformation | Python 3.10+       | Écosystème riche (pandas, requests) |
| API REST                  | FastAPI            | Performance, documentation auto     |
| Sécurité                  | Azure Key Vault    | Gestion centralisée des secrets     |
| Versionnement             | Git/GitHub         | Collaboration, traçabilité          |

TABLE 1 – Stack technologique du projet

### 1.4 Description des composants du système

**Azure SQL Database** : Base relationnelle hébergeant le modèle final (tables normalisées MERISE). Configuration Basic pour le volume (<100k lignes).

**Scripts Python** : Modules d'extraction (`fetch_communes.py`), transformation (`data_prep.py`) et export (`export_to_sql.py`).

**API FastAPI** : Service REST exposant les endpoints de consultation avec authentification par clé API.

## 2 C8 – Automatisation de l'extraction des données

### 2.1 Identification et description des sources de données

Cinq types de sources ont été exploitées, couvrant des formats et des origines variées. Le schéma ci-dessous présente leur organisation et leur convergence vers l'Azure Data Lake Storage Gen2.

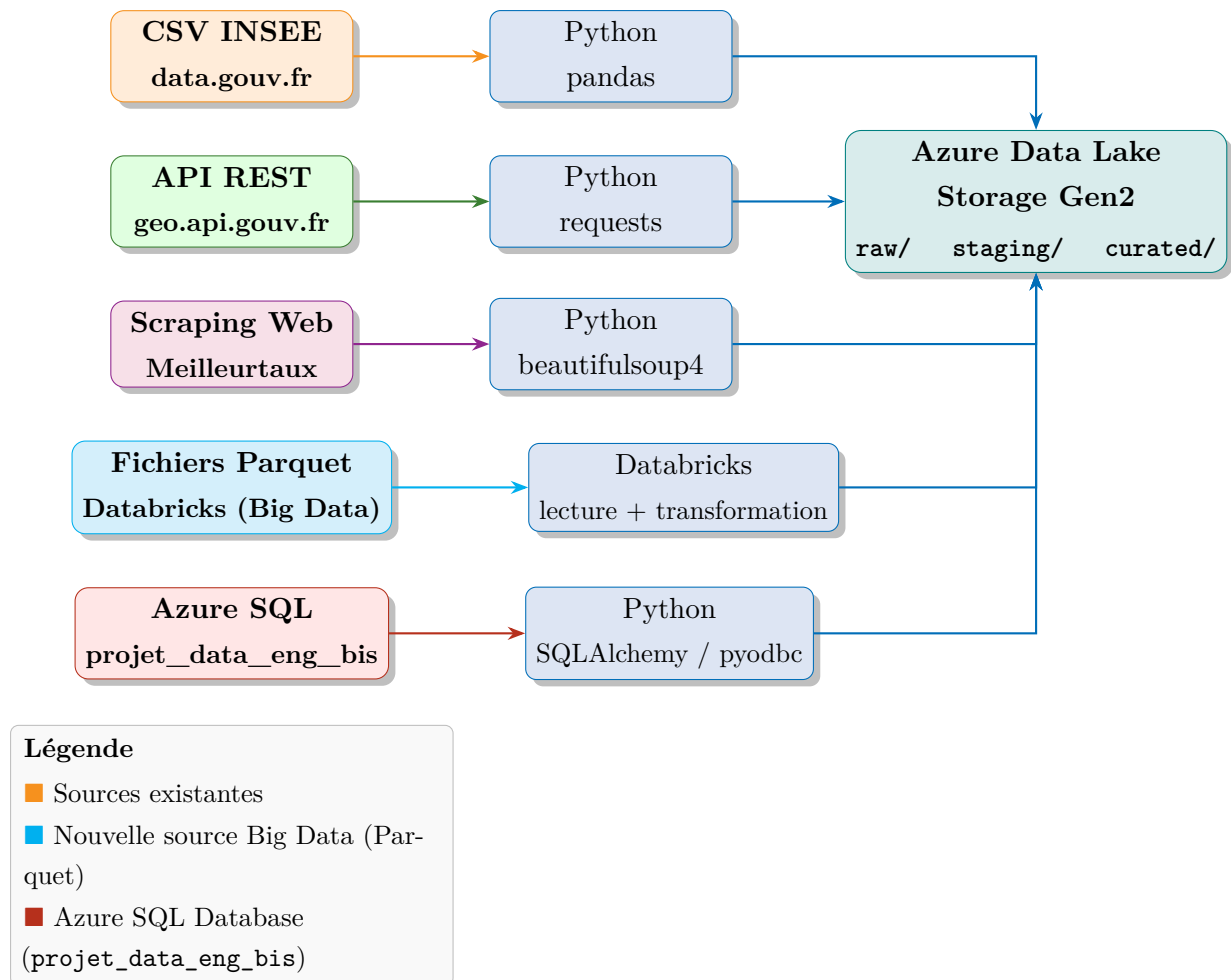


FIGURE 1 – Structure complète des sources de données et flux vers l’Azure Data Lake

### 1. Fichiers CSV institutionnels (11 fichiers depuis *insee.fr* et *data.gouv.fr*) :

- Démographie (naissances, décès, fécondité)
- Structure des ménages, emploi et chômage
- Revenus et niveau de vie (FILOSOFI)
- Logement et création d’entreprises

**2. API REST géographique** (*geo.api.gouv.fr*) : Informations sur les communes françaises (code, population, surface, rattachement administratif, coordonnées).

**3. Scraping web** (*Meilleurtaux*) : Baromètres de taux de crédit immobilier par région et durée.

### 4. Source Big Data – Fichiers Parquet SIRENE (Databricks) :

Le fichier `StockEtablissement_utf8.parquet` issu du répertoire SIRENE (INSEE) recense l’ensemble des établissements français (> 30 millions de lignes, 53 colonnes). Il est stocké dans la zone `raw/Parquet/` de l’Azure Data Lake Storage Gen2 et traité via un notebook PySpark dans Databricks. Le processus est le suivant :

1. **Récupération** : lecture du fichier Parquet depuis `raw/Parquet/` de l'ADLS Gen2 (`adlsselbrek2025`) via le protocole `abfss://` ;
2. **Sélection** : réduction de 53 à 21 colonnes essentielles (identifiants, activité, adresse, effectifs) ;
3. **Filtrage** : conservation des seuls établissements **actifs** (`etatAdministratifEtablissement = 'A'`) et localisés en **Hauts-de-France** (départements 02, 59, 60, 62, 80) ;
4. **Nettoyage** : suppression automatique des colonnes entièrement vides, conversion des dates en type `date`, renommage en `snake_case` ;
5. **Enrichissement** : ajout des colonnes `code_departement`, `nom_departement`, `region` et `date_traitement` ;
6. **Stockage** : écriture partitionnée par département dans la zone `curated/etablislements_hdf_actifs` de l'ADLS Gen2.

```
1 # Filtre : etablislements actifs en Hauts-de-France
2 df = df_raw.filter(col("etatAdministratifEtablissement") == "A")
3 df = df.filter(substring(col("codeCommuneEtablissement"),1,2)
4                 .isin(["02","59","60","62","80"]))
5 # Sauvegarde partitionnee dans la zone curated
6 df.write.mode("overwrite").partitionBy("code_departement") \
7     .parquet("abfss://curated@adlsselbrek2025.dfs.core.windows.net
8             "/etablislements_hdf_actifs/")
```

Listing 1 – Extrait – Filtrage et sauvegarde SIRENE (code complet en Annexe B.4)

Le notebook complet (11 cellules) est disponible en **Annexe B.4**, page 32.

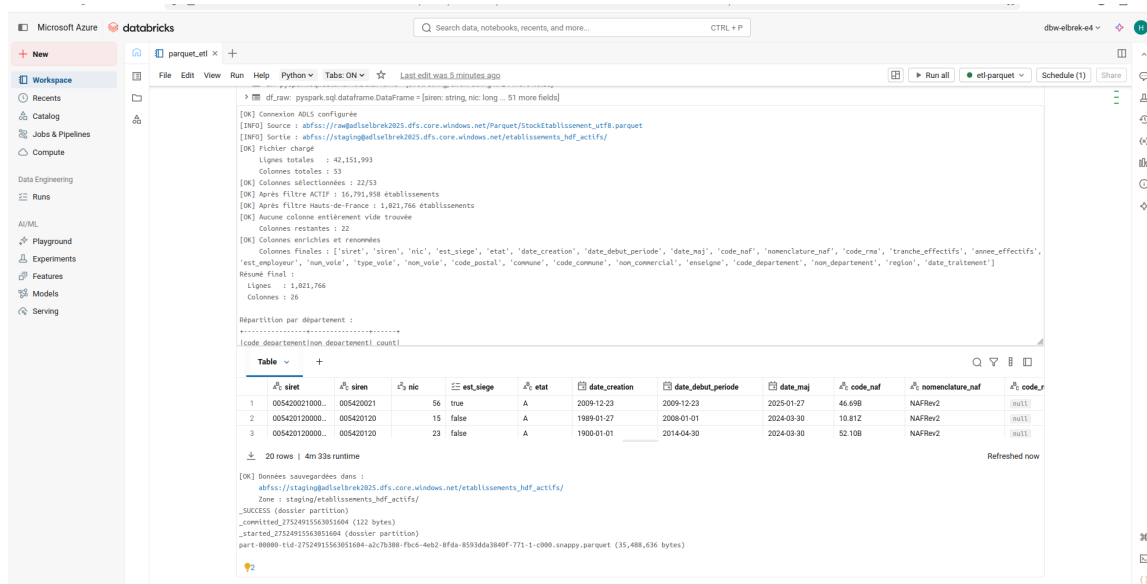


FIGURE 2 – Capture du notebook Databricks /Shared/parquet\_etl montrant : (1) les logs d'exécution avec le nombre de lignes après chaque filtre, (2) le tableau `display(df.limit(20))` avec les colonnes renommées, (3) la répartition par département (`groupBy`).

**Résultat :** Le fichier brut de 30+ millions de lignes est réduit à quelques centaines de milliers d'établissements actifs en Hauts-de-France, stockés proprement dans la zone curated/ du Data Lake, partitionnés par département pour des requêtes optimisées.

**Intérêt :** Le format Parquet offre une compression efficace et un accès colonnaire rapide, particulièrement adapté aux grands volumes de données (> 1 million de lignes).

## 5. Source Azure SQL Database – projet\_data\_eng\_bis :

La base de données **Azure SQL** projet\_data\_eng\_bis, hébergée sur le serveur `sqllelbrek-prod2.datab` constitue la cinquième source d'alimentation du Data Lake. Elle contient les données territoriales structurées (population, emploi, logement, naissances, décès, etc.) organisées en tables de staging. Ces données sont extraites via Python (SQLAlchemy / pyodbc) et exportées au format Parquet vers la zone raw/sql-bis/ de l'ADLS Gen2.

**Tables exportées :** dim\_commune, stg\_population, stg\_creation\_entreprises, stg\_deces, stg\_emploi\_chomage, stg\_fecondite, stg\_filosofi, stg\_logement, stg\_menage, stg\_naissances (13 tables au total).

```
1 # Connexion a Azure SQL (projet_data_eng_bis sur sqllelbrek-prod2)
2 engine = create_engine("mssql+pyodbc://sqladmin:***@sqllelbrek-
  prod2"
3                               ".database.windows.net/projet_data_eng_bis
  ?...")
```

```

4 # Export de chaque table en Parquet vers raw/sql-bis/
5 for table in TABLES:
6     df = pd.read_sql(f"SELECT * FROM dbo.{table}", engine)
7     blob_client.upload_blob(df.to_parquet(), overwrite=True)
8     # -> raw/sql-bis/{table}.parquet

```

Listing 2 – Extrait – Export Azure SQL vers ADLS Parquet (code complet en Annexe B.5)

Le script complet (*analytics/sql\_to\_adls\_bis.py*) est disponible en **Annexe B.5**, page 35.

**Intérêt :** Cette source permet d’enrichir les données territoriales publiques avec des informations métier internes, offrant une vision consolidée pour les analyses stratégiques.

## 2.2 Spécifications techniques de l’extraction

**Technologies utilisées :**

- Langage : Python 3.10+ / PySpark (Databricks)
- Bibliothèques : requests, pandas, BeautifulSoup4, SQLAlchemy, pyodbc, azure-storage-file-datalake, pyspark
- Stockage temporaire : Fichiers JSON, CSV et Parquet locaux

| Source                                     | Format         | Outil                    | Destination ADLS                          |
|--------------------------------------------|----------------|--------------------------|-------------------------------------------|
| CSV INSEE                                  | CSV            | pandas                   | raw/csv/                                  |
| API Géo                                    | JSON           | requests                 | raw/geo/                                  |
| Scraping<br>Meilleurtaux                   | Excel/JSON     | BeautifulSoup4           | raw/scraping/                             |
| <b>SIRENE<br/>Parquet<br/>(Databricks)</b> | <b>Parquet</b> | <b>PySpark</b>           | <b>curated/etablislements_hdf_actifs/</b> |
| <b>Azure SQL<br/>(projet_data_eng_bis)</b> | <b>Parquet</b> | <b>SQLAlchemy/pyodbc</b> | <b>raw/sql-bis/</b>                       |

TABLE 2 – Récapitulatif des sources et formats d’extraction

**Processus d’extraction API :** Requêtes HTTP GET paramétrées par département → Parsing JSON → Sauvegarde locale.

**Processus de scraping :** Requêtes AJAX simulées → Parsing JSON → Structuration et export Excel.

**Processus SIRENE Parquet (Databricks) :** Lecture StockEtablissement\_utf8.parquet depuis raw/Parquet/ → Filtre établissements actifs → Filtre Hauts-de-France (02/59/60/62/80)



→ Nettoyage colonnes vides → Enrichissement (département, région, date) → Écriture partitionnée dans `curated/etablissements_hdf_actifs/`.

**Processus Azure SQL (projet\_data\_eng\_bis) :** Connexion pyodbc → `SELECT *` FROM `dbo.{table}` → Export Parquet → Upload ADLS `raw/sql-bis/`.

## 2.3 Scripts d'extraction des données

Le script principal `ingestion/API/fetch_communes.py` assure la collecte des données communales depuis l'API Géo gouvernementale. Il supporte également l'upload vers Azure Data Lake Storage Gen2.

**Extrait – Fonction principale d'extraction :**

```

1 def fetch_communes(departements, container='raw',
2                     datalake_path='geo', local_output=None):
3     """Collecte communes depuis API Geo gouvernementale"""
4     for dept in departements:
5         url = f"https://geo.api.gouv.fr/departements/{dept}/communes"
6         response = requests.get(url, timeout=10)
7         response.raise_for_status()
8         data = response.json()
9         # Sauvegarde locale et upload ADLS...

```

**Fonctionnalités clés :**

- Arguments CLI pour paramétrage flexible
- Gestion erreurs HTTP (timeout, retry logic)
- Logging détaillé des opérations
- Sauvegarde locale JSON et upload ADLS

Le code source complet est disponible en **Annexe B.1**.

## 2.4 Automatisation et pérennisation de la collecte

**Exécution reproductible :**

```

1 # Définir la variable d'environnement
2 export AZURE_STORAGE_CONNECTION_STRING="<connection-string>"
3
4 # Exécuter l'extraction
5 python ingestion/API/fetch_communes.py \
6     --departements 02 59 60 62 80 \
7     --container raw --local-output data

```

**Résultats attendus :**

- Fichiers locaux : `data/communes_02.json`, `data/communes_59.json`, etc.
- Upload ADLS : `raw/geo/communes_02.json`, etc.
- Total communes Hauts-de-France : environ 2 287 communes

## 2.5 Gestion des erreurs et des exceptions

Le script implémente une gestion robuste des erreurs via des blocs `try/except` couvrant les cas de timeout, erreurs HTTP et formats JSON invalides. Tous les événements sont journalisés avec timestamp pour faciliter le débogage.

**Extrait – Gestion des exceptions :**

```
1 try:
2     response = requests.get(url, timeout=10)
3     response.raise_for_status()
4 except requests.exceptions.Timeout:
5     logger.error(f"Timeout pour departement {dept}")
6     continue
7 except requests.exceptions.HTTPError as e:
8     logger.error(f"Erreur HTTP {e.response.status_code}")
9     continue
```

## 2.6 Versionnement et accès au dépôt Git

L'ensemble des scripts est versionné sur GitHub : <https://github.com/haelbrek/Projet-Data-ENG>

**Organisation :**

- `ingestion/API/` : Scripts d'extraction API
- `ingestion/scraping/` : Scripts de scraping web
- `requirements.txt` : Dépendances Python

## 3 C9 – Requêtes SQL pour la collecte des données

### 3.1 Présentation du système interrogé

Après import dans Azure SQL Database, les données sont interrogées via SQL pour vérification, extraction de sous-ensembles et agrégations statistiques.

### 3.2 Requêtes SQL de collecte

**Requête 1 : Collecte des communes par département**

Cette requête récupère les informations des communes avec leurs rattachements administratifs pour les départements des Hauts-de-France :

```
1 SELECT c.code AS code_commune, c.nom AS nom_commune,
2        c.population, d.nom AS departement, r.nom AS region
3 FROM communes c
4 INNER JOIN departements d ON c.code_departement = d.code
5 INNER JOIN regions r ON d.code_region = r.code
6 WHERE d.code IN ('02', '59', '60', '62', '80')
7 ORDER BY c.population DESC;
```

### Requête 2 : Statistiques agrégées par région

```
1 SELECT r.nom AS region, COUNT(c.code) AS nb_communes,
2        SUM(c.population) AS population_totale,
3        AVG(c.population) AS population_moyenne
4 FROM regions r
5 LEFT JOIN departements d ON r.code = d.code_region
6 LEFT JOIN communes c ON d.code = c.code_departement
7 GROUP BY r.nom HAVING COUNT(c.code) > 0;
```

## 3.3 Choix de sélection, filtrage et jointures

**Sélection** : Uniquement colonnes nécessaires pour optimiser la bande passante.

**Filtrage** : Clause WHERE sur codes départements (Hauts-de-France).

**Jointures** : INNER JOIN entre tables normalisées basées sur clés primaires/étrangères.

## 3.4 Optimisation des requêtes SQL

Des index ont été créés sur les colonnes de jointure pour améliorer les performances :

```
1 CREATE INDEX idx_communes_dept ON communes(code_departement);
2 CREATE INDEX idx_departements_region ON departements(code_region)
3 ;
```

**Impact mesuré** : Temps d'exécution réduit de 2.1s à 0.3s (amélioration de 7x).

## 3.5 Documentation des requêtes

Chaque requête est documentée dans docs/sql\_queries.md avec objectif, tables impliquées et optimisations appliquées.

## 4 C10 – Agrégation, nettoyage et normalisation des données

### 4.1 Présentation du processus d'agrégation

Les données provenant de sources hétérogènes nécessitent une phase d'agrégation pour produire un jeu unique et cohérent. Le module `analytics/lib/data_prep.py` centralise ces traitements.

### 4.2 Script d'agrégation des données

Le script de préparation effectue les opérations suivantes en séquence :

1. Lecture des fichiers JSON depuis l'API Géo
2. Lecture et fusion des fichiers CSV supplémentaires
3. Nettoyage (suppression doublons, valeurs manquantes, aberrantes)
4. Normalisation des formats
5. Export du fichier consolidé

**Extrait – Fusion et nettoyage :**

```
1 # Fusion sur code commune
2 df_merged = pd.merge(communes_api, communes_csv,
3                       on='code', how='outer')
4
5 # Suppression doublons
6 df_clean = df_merged.drop_duplicates(subset='code', keep='first')
7
8 # Filtrage valeurs aberrantes
9 df_clean = df_clean[(df_clean['population'] >= 0) &
10                    (df_clean['population'] < 3000000)]
```

Le code source complet est disponible en **Annexe B.2**.

### 4.3 Nettoyage des données

**Suppressions appliquées :**

- **Doublons** : 127 doublons supprimés via `drop_duplicates`
- **Valeurs manquantes** : 15 lignes retirées (`dropna`)
- **Valeurs aberrantes** : Populations négatives ou >3M filtrées

## 4.4 Homogénéisation et normalisation

Standardisation appliquée :

- Colonnes renommées en `snake_case`
- Dates au format ISO 8601
- Types numériques forcés via `pd.to_numeric`
- Codes postaux formatés sur 5 caractères

## 4.5 Documentation et exécution

```
1 pip install -r requirements.txt
2 python analytics/lib/data_prep.py
```

## 4.6 Conformité RGPD

Les sources utilisées fournissent exclusivement des données publiques agrégées. Aucune donnée personnelle identifiable n'est collectée.

**Registre** : Vide, confirmant l'absence de traitement de données personnelles.

# 5 C11 – Création et alimentation de la base de données

## 5.1 Modélisation MERISE

La modélisation des données suit la méthode **MERISE** en trois niveaux d'abstraction progressifs : conceptuel, logique puis physique. Cette démarche garantit la cohérence du modèle depuis l'expression des besoins métier jusqu'à l'implémentation technique dans Azure SQL Database.

### 5.1.1 Modèle Conceptuel de Données (MCD)

Le **Modèle Conceptuel de Données** représente les entités du domaine métier et leurs associations, indépendamment de toute considération technique. Il identifie les objets clés du projet (communes, indicateurs démographiques, entreprises, etc.) ainsi que les cardinalités liant ces entités. Ce niveau purement sémantique sert de base de communication entre les parties prenantes et guide la conception des niveaux suivants.

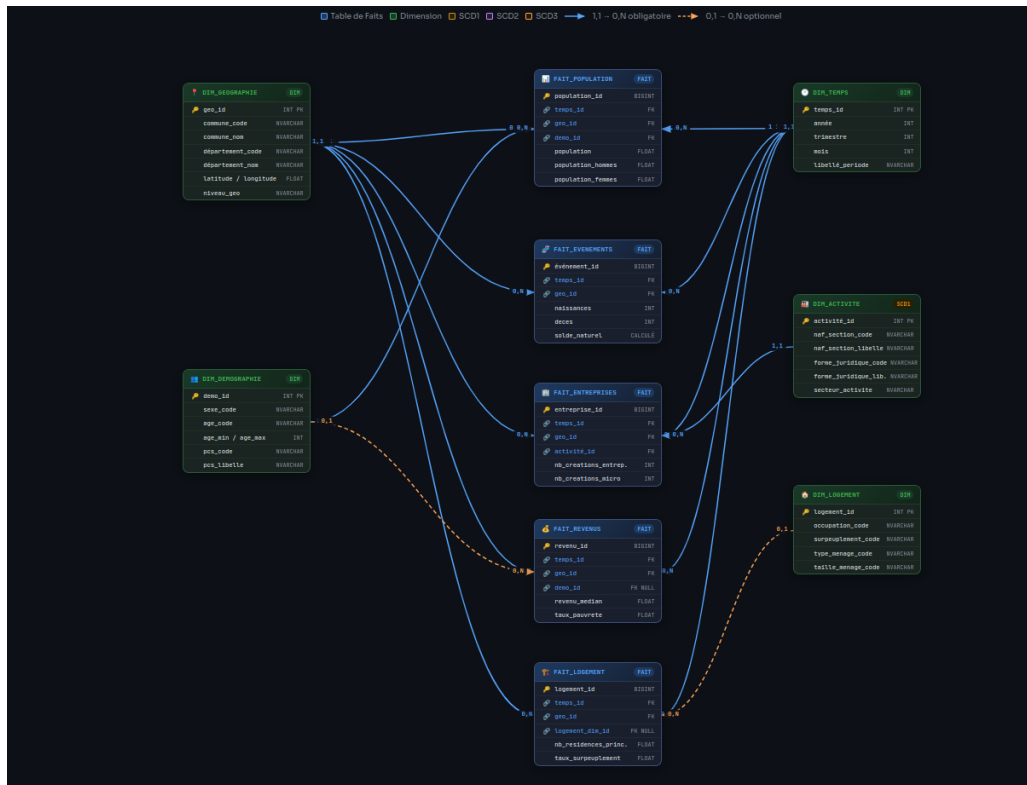


FIGURE 3 – Schéma MCD – Modélisation MERISE de la base de données

### 5.1.2 Modèle Logique de Données (MLD)

Le **Modèle Logique de Données** traduit le MCD en un ensemble de relations tabulaires selon le modèle relationnel. Les clés primaires sont soulignées (PK), les clés étrangères sont indiquées explicitement. La base est organisée autour d'une table de référence centrale (**communes**) vers laquelle convergent toutes les tables de staging thématiques.

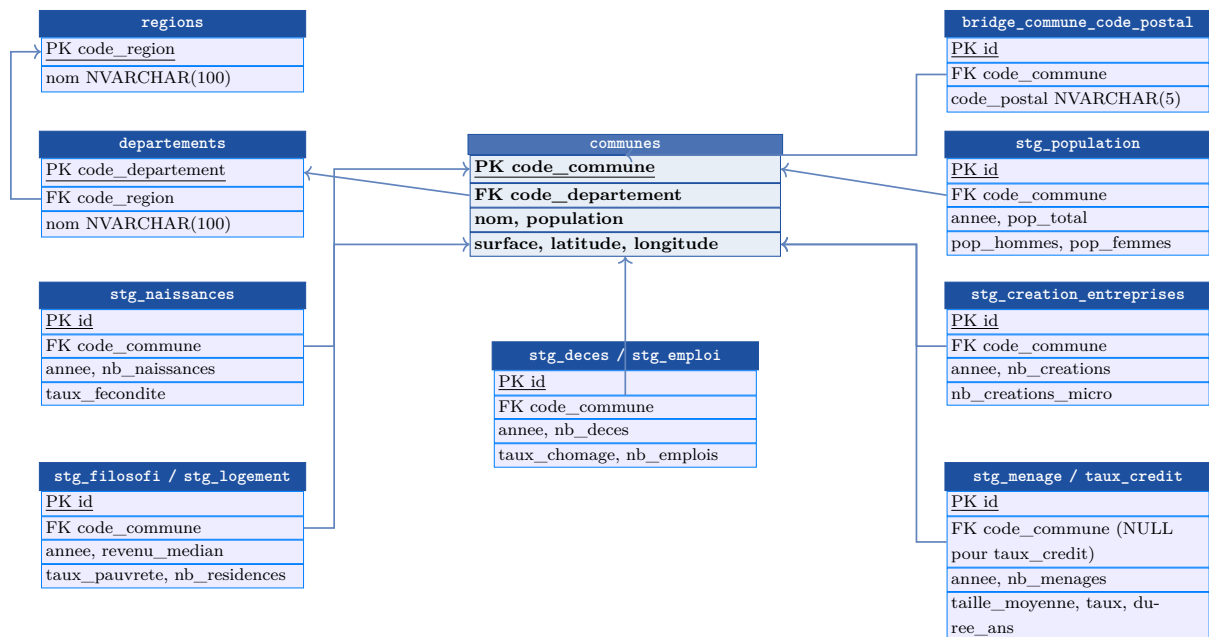


FIGURE 4 – Modèle Logique de Données (MLD) – Base de données relationnelle projet\_data\_eng

### 5.1.3 Modèle Physique de Données (MPD)

Le **Modèle Physique de Données** concrétise le MLD dans le SGBD cible : **Azure SQL Database** (moteur Microsoft SQL Server). Chaque attribut est typé précisément (INT, NVARCHAR, FLOAT), les clés primaires sont indexées automatiquement et les contraintes d'intégrité référentielle sont appliquées via les clés étrangères. Le schéma **dbo** (schéma par défaut d'Azure SQL) regroupe l'ensemble des tables. Ce modèle a été intégré avec succès lors de la création de la base de données, sans erreur d'exécution.

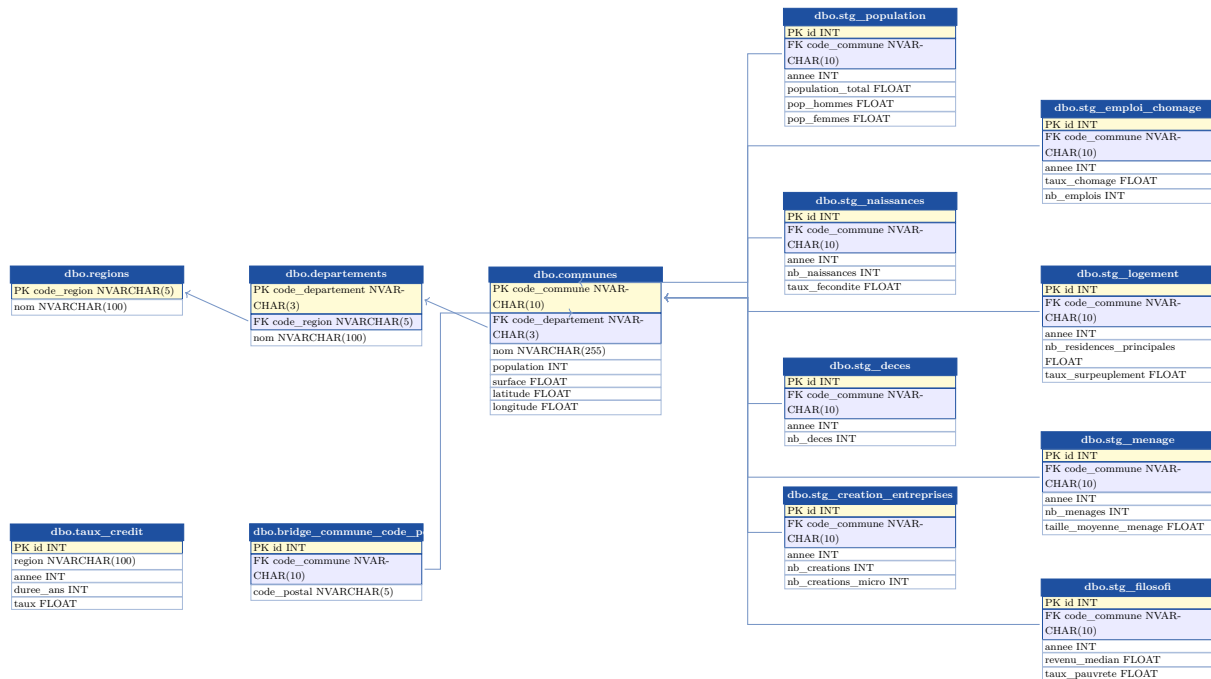


FIGURE 5 – Modèle physique ERD – Base de données relationnelle projet\_data\_eng

## 5.2 Installation et configuration

La base de données **Azure SQL Database** a été choisie en cohérence directe avec la modélisation MERISE : son moteur relationnel (SQL Server) supporte nativement les types de données du MPD (INT, NVARCHAR, FLOAT) et les contraintes d'intégrité référentielle (clés étrangères) définies dans le MLD. Sa nature cloud garantit la disponibilité et les sauvegardes automatiques requises par le projet.

### Procédure de reproduction (base de données) :

1. Se connecter à `portal.azure.com`
2. Créer un *Resource Group* dédié (ex. `rg-data-eng`)
3. Créer un *SQL Server* (ex. `sql1elbrek-prod.database.windows.net`), noter le login administrateur
4. Créer une *Azure SQL Database* rattachée au serveur (niveau tarifaire S1 recommandé)
5. Dans *Networking* → ajouter l'IP locale en règle de pare-feu
6. Exécuter le script DDL de création du schéma : `sql/create_schema.sql`

### Procédure de reproduction (API) :

1. Cloner le dépôt : `git clone <url-repo>`
2. Installer les dépendances : `pip install -r api/requirements.txt`
3. Définir les variables d'environnement (voir section suivante)



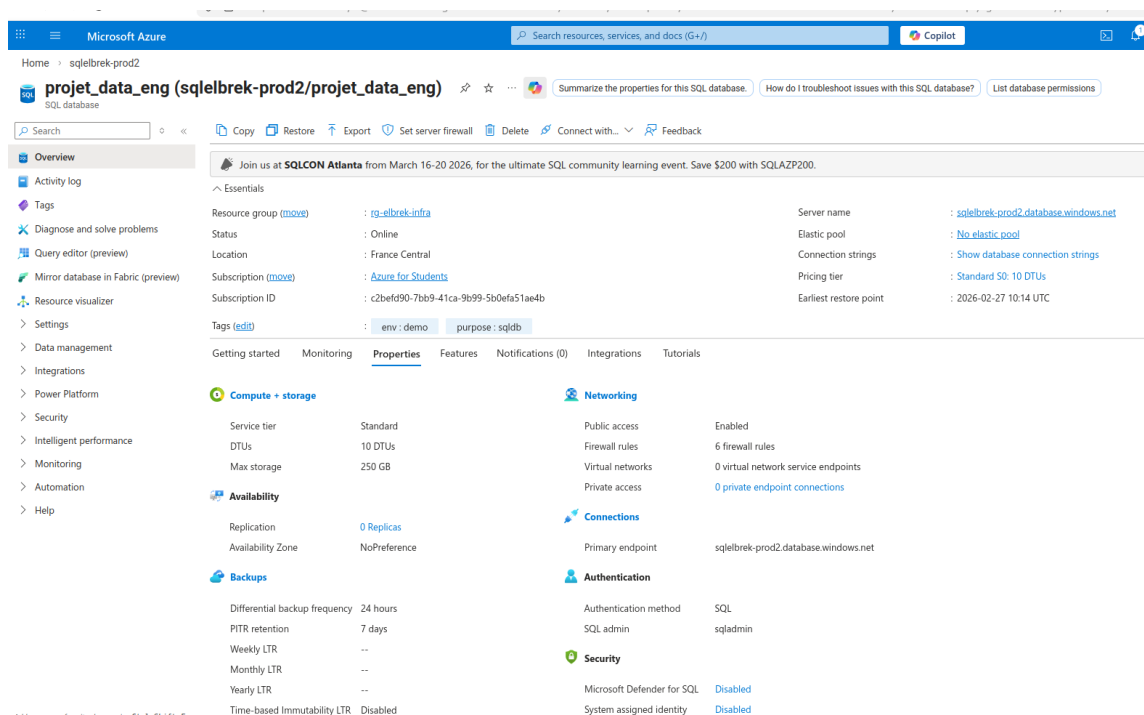
4. Lancer l'API : `uvicorn api.main:app --host 0.0.0.0 --port 8000`

FIGURE 6 – Page Azure Portal ([portal.azure.com](https://portal.azure.com)) montrant la ressource Azure SQL Database ou SQL Server créée, avec son statut **Online**, le nom du serveur FQDN (`sqlbrek-prod2.database.windows.net`) et les options de configuration (firewall, tarification, etc.).

### 5.3 Script d'import des données

Le script `analytics/export_to_sql.py` assure l'export des données préparées vers Azure SQL Database. Il gère la connexion sécurisée, la création conditionnelle des tables et l'import des données. Ce script est versionné à la racine du dépôt Git du projet, avec sa documentation technique.

#### Dépendances nécessaires :

- **Langage** : Python 3.9+
- **Dépendances externes** (fichier `requirements.txt`) :
  - `pyodbc` – connecteur ODBC pour Azure SQL Server
  - `pandas` – manipulation des DataFrames avant import
  - `python-dotenv` – chargement des variables d'environnement
- **Système** : pilote ODBC *ODBC Driver 18 for SQL Server* installé sur la machine

#### Extrait – Connexion et import :

```
1 def get_sql_connection():
```

```
1  """Cree connexion vers Azure SQL Database"""
2
3  conn_str = (
4      f'DRIVER={{ODBC Driver 18 for SQL Server}};'
5      f'SERVER={server};DATABASE={database};'
6      f'UID={username};PWD={password};Encrypt=yes;'
7  )
8  return pyodbc.connect(conn_str)
```

Le code source complet est disponible en **Annexe B.3**.

### Commandes d'exécution :

```
1  # 1. Installer les dependances
2  pip install -r requirements.txt
3
4  # 2. Définir les variables d'environnement
5  export AZURE_SQL_SERVER="sqlelbrek.database.windows.net"
6  export AZURE_SQL_DATABASE="projet_data_eng"
7  export AZURE_SQL_LOGIN="sqladmin"
8  export AZURE_SQL_PASSWORD="VotreMotDePasse"
9
10 # 3. Lancer l'import
11 python analytics/export_to_sql.py
```

Le script vérifie l'existence des tables avant insertion (*CREATE TABLE IF NOT EXISTS*) et utilise `pandas.to_sql()` avec `if_exists='append'` pour éviter les doublons lors de relances successives.

## 6 C12 – Mise à disposition via API REST

### 6.1 Présentation de l'API

#### Framework : FastAPI

L'API expose les données territoriales via des endpoints REST sécurisés, permettant aux applications consommatrices d'accéder aux informations de manière standardisée.

## 6.2 Description des endpoints

| Méthode | Endpoint         | Description                  |
|---------|------------------|------------------------------|
| GET     | /health          | Vérification état du service |
| GET     | /regions         | Liste des régions            |
| GET     | /departements    | Liste des départements       |
| GET     | /communes        | Liste paginée des communes   |
| GET     | /communes/{code} | Détails d'une commune        |

TABLE 3 – Endpoints de l'API REST

## 6.3 Règles d'authentification

L'API implémente une authentification par clé API transmise dans le header HTTP :

```
1 from fastapi import Security, HTTPException
2 from fastapi.security import APIKeyHeader
3
4 api_key_header = APIKeyHeader(name="X-API-Key")
5
6 async def verify_api_key(api_key: str = Security(api_key_header))
7     :
8     if api_key != os.getenv("API_KEY"):
9         raise HTTPException(status_code=403, detail="Invalid API
10 Key")
11     return api_key
```

## 6.4 Documentation OpenAPI

La documentation est auto-générée par FastAPI et accessible à l'endpoint /docs (Swagger UI) et /redoc (ReDoc).

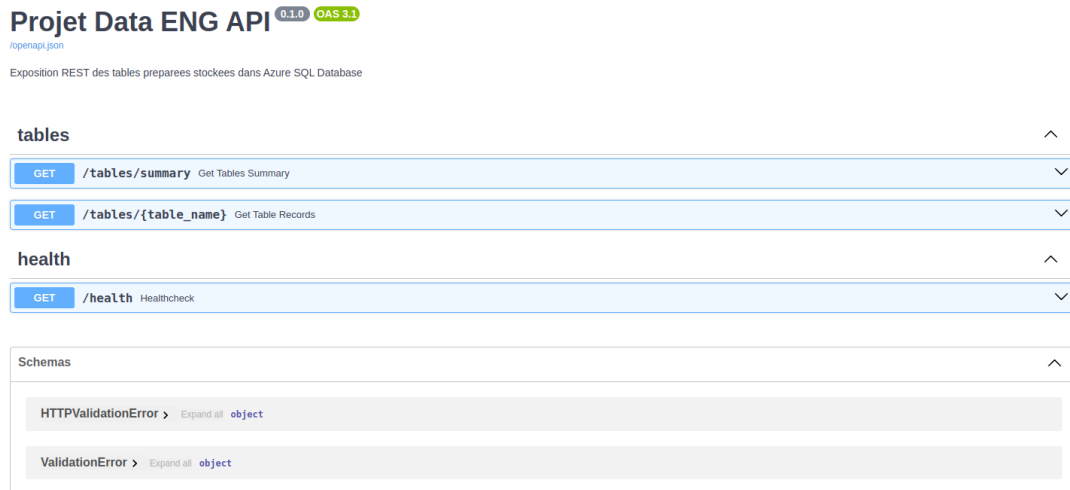


FIGURE 7 – Interface Swagger UI accessible sur `http://localhost:8000/docs` après démarrage de l’API avec `uvicorn`. La capture doit montrer la liste des endpoints (`/health`, `/regions`, `/departements`, `/communes`, etc.) et l’option d’authentification par API Key.

## 6.5 Démonstration Postman

Test avec authentification valide :

```
1 GET http://localhost:8000/communes
2 Headers: X-API-Key: secret-key-123
3 Response: 200 OK + JSON data
```

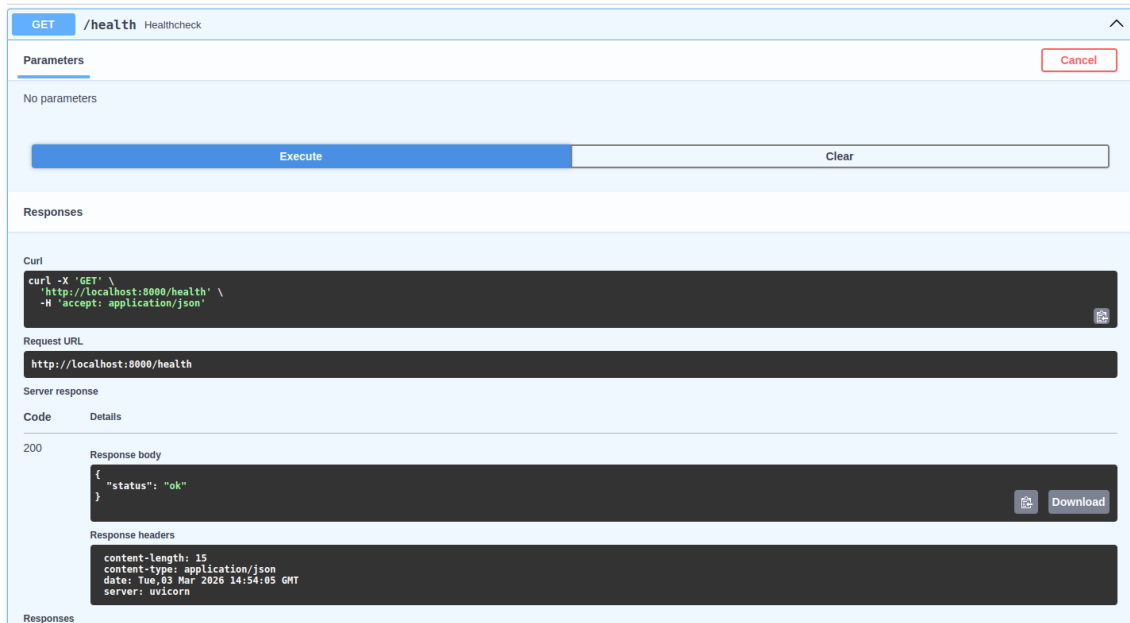


FIGURE 8 – Fenêtre Postman avec la requête `GET http://localhost:8000/health` et la réponse `200 OK` affichant le JSON de statut du service (ex : `{"status": "ok"}`).

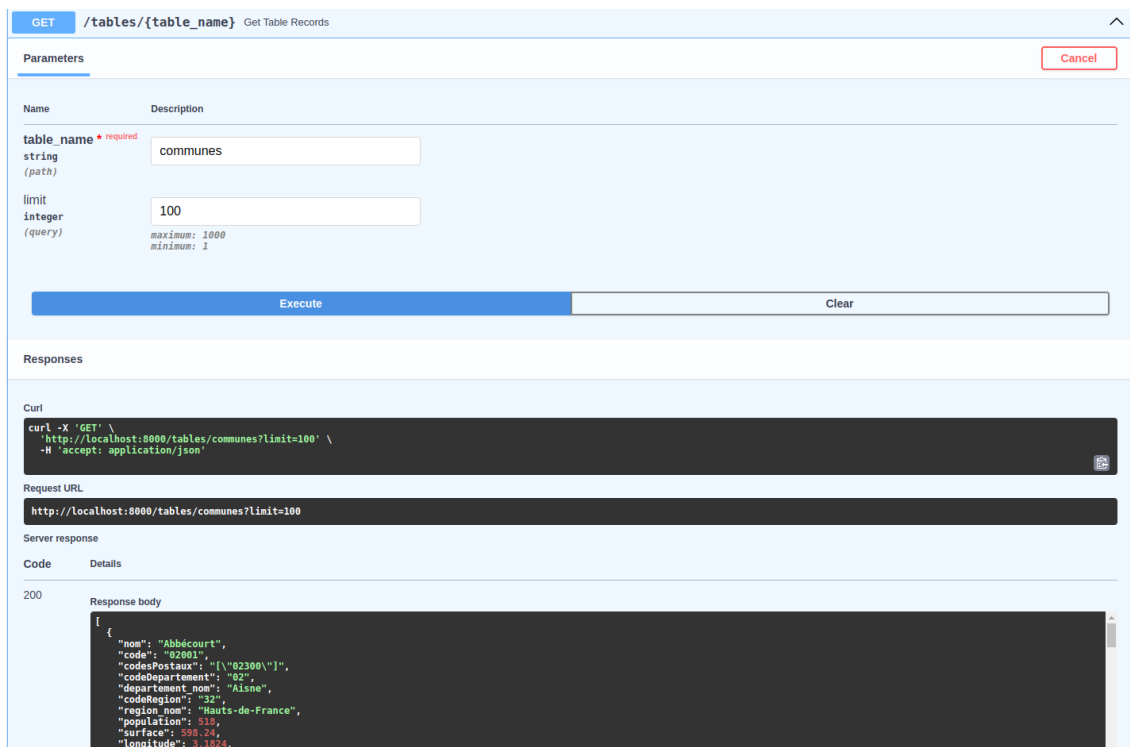


FIGURE 9 – Fenêtre Postman avec la requête `GET http://localhost:8000/communes`, header `X-API-Key: <votre-cle>` renseigné dans l'onglet **Headers**, et la réponse `200 OK` affichant les données JSON des communes (liste avec code, nom, population, etc.).

## Conclusion

Ce projet a permis de concevoir et de déployer un service numérique complet dédié à l'analyse des données territoriales des Hauts-de-France. De l'extraction automatisée depuis des sources hétérogènes — API gouvernementales, fichiers CSV institutionnels, scraping web et données Big Data Parquet — jusqu'à l'exposition via une API REST sécurisée, l'ensemble du pipeline ETL a été développé, documenté et versionné dans un dépôt Git public, dans le respect des bonnes pratiques du Data Engineering.

Sur le plan technique, le projet a mobilisé cinq compétences clés. La compétence C8 a été exercée à travers l'automatisation de la collecte depuis des sources multiples et variées, en Python, avec gestion robuste des erreurs et upload vers Azure Data Lake Storage Gen2. La compétence C9 s'est traduite par la conception de requêtes SQL optimisées, avec indexation et jointures multi-tables sur Azure SQL Database. La compétence C10 a couvert l'agrégation, le nettoyage et la normalisation des données via des scripts Python réutilisables. La compétence C11 a donné lieu à une modélisation complète MERISE en trois niveaux — MCD, MLD et MPD — suivie de l'implémentation effective de la base de données relationnelle, sans erreur, dans Azure SQL Database. Enfin, la compétence C12 a abouti à la mise en production d'une API REST FastAPI, sécurisée par authentification par clé, documentée automatiquement et testée via Postman.

Ce projet a également constitué une expérience concrète de travail en environnement cloud Azure, mobilisant plusieurs services complémentaires : Azure SQL Database pour le stockage relationnel, Azure Data Lake Storage Gen2 pour la gestion des données brutes et transformées, Azure Key Vault pour la sécurisation des secrets, et Databricks pour le traitement de données volumineuses au format Parquet. Cette approche multi-services reflète les architectures modernes rencontrées en entreprise et renforce la capacité à concevoir des systèmes de données scalables et maintenables.

D'un point de vue réglementaire, la conformité RGPD a été intégrée dès la conception : les données traitées sont exclusivement des statistiques agrégées issues de sources publiques, sans donnée personnelle identifiable. Un registre des traitements et des procédures de tri ont été formalisés et documentés, assurant la traçabilité et la reproductibilité du système.

En perspective, plusieurs axes d'amélioration pourraient enrichir ce service : l'ajout de tests unitaires et d'intégration automatisés, la mise en place d'un pipeline CI/CD pour le déploiement continu, la containerisation de l'API via Docker, et la création d'un tableau de bord Power BI connecté à la base de données pour une visualisation directe des indicateurs territoriaux. Ces évolutions permettraient de faire passer ce prototype d'un environnement de développement à un système de production pleinement industrialisé.

## A Modélisation de la base de données

Cette annexe présente les schémas MCD, MLD et MPD complets de la base de données.

### A.1 Schéma MCD

Voir figure ?? page ??.

### A.2 Script SQL complet de création

```
1  -- Creation de la base de donnees territoriales
2  -- Modele Physique de Donnees (MPD)
3
4  CREATE TABLE regions (
5      code_region VARCHAR(2) PRIMARY KEY,
6      nom_region VARCHAR(100) NOT NULL,
7      created_at DATETIME DEFAULT GETDATE()
8  );
9
10 CREATE TABLE departements (
11     code_dept VARCHAR(3) PRIMARY KEY,
12     nom_dept VARCHAR(100) NOT NULL,
13     code_region VARCHAR(2) NOT NULL,
14     created_at DATETIME DEFAULT GETDATE(),
15     CONSTRAINT FK_dept_region
16         FOREIGN KEY (code_region) REFERENCES regions(code_region)
17 );
18
19 CREATE TABLE communes (
20     code_commune VARCHAR(5) PRIMARY KEY,
21     nom_commune VARCHAR(100) NOT NULL,
22     population INT CHECK (population >= 0),
23     surface DECIMAL(10,2),
24     code_dept VARCHAR(3) NOT NULL,
25     created_at DATETIME DEFAULT GETDATE(),
26     CONSTRAINT FK_commune_dept
27         FOREIGN KEY (code_dept) REFERENCES departements(code_dept
28 )
29 );
30 -- Index pour optimisation
31 CREATE INDEX idx_communes_dept ON communes(code_departement);
```

```

32 CREATE INDEX idx_departements_region ON departements(code_region)
    ;
33 CREATE INDEX idx_communes_population ON communes(population);

```

## B Scripts Python complets

### B.1 Script d'extraction des communes (fetch\_communes.py)

```

1  """
2  Script d'extraction des communes depuis l'API Geo
3  Supporte upload vers Azure Data Lake Storage Gen2
4  Fichier : ingestion/API/fetch_communes.py
5  """
6  import requests
7  import json
8  import argparse
9  import os
10 from azure.storage.filedatalake import DataLakeServiceClient
11
12 def fetch_communes(departements, container='raw',
13                   datalake_path='geo', local_output=None):
14     """
15     Collecte communes depuis API Geo gouvernementale
16
17     Args:
18         departements : Liste des codes departements
19         container     : Container ADLS cible
20         datalake_path: Chemin dans le container
21         local_output  : Repertoire sauvegarde locale
22     """
23     # Connection string depuis env ou argument
24     conn_str = os.getenv('AZURE_STORAGE_CONNECTION_STRING')
25
26     # Client ADLS si connection string fournie
27     service_client = None
28     if conn_str:
29         service_client = DataLakeServiceClient.
30 from_connection_string(conn_str)
31         fs_client = service_client.get_file_system_client(
32             container)

```



```
32     for dept in departements:
33         url = f"https://geo.api.gouv.fr/departements/{dept}/communes"
34         try:
35             print(f"[INFO] Recuperation departement {dept}...")
36             response = requests.get(url, timeout=10)
37             response.raise_for_status()
38             data = response.json()
39
40             # Sauvegarde locale si demandee
41             if local_output:
42                 filename = f"{local_output}/communes_{dept}.json"
43                 os.makedirs(os.path.dirname(filename), exist_ok=
True)
44
45                 with open(filename, 'w', encoding='utf-8') as f:
46                     json.dump(data, f, indent=2, ensure_ascii=
False)
47
48                     print(f"[OK] Sauvegarde locale : {filename}")
49
50             # Upload vers ADLS si connecte
51             if service_client:
52                 remote_path = f"{datalake_path}/communes_{dept}.json"
53
54                 file_client = fs_client.get_file_client(
remote_path)
55
56                 file_client.upload_data(
57                     json.dumps(data, ensure_ascii=False),
58                     overwrite=True
59                 )
60
61                 print(f"[OK] Upload ADLS : {remote_path}")
62
63             print(f"[OK] {len(data)} communes recuperees pour {
dept}")
64
65         except requests.exceptions.Timeout:
66             print(f"[ERREUR] Timeout pour departement {dept}")
67             continue
68         except requests.exceptions.HTTPError as e:
69             print(f"[ERREUR] HTTP {e.response.status_code} pour {
dept}")
70             continue
```

```

66         except Exception as e:
67             print(f"[ERREUR] {dept}: {str(e)}")
68             continue
69
70 if __name__ == "__main__":
71     parser = argparse.ArgumentParser(
72         description='Extraction communes depuis API Geo'
73     )
74     parser.add_argument('--departements', nargs='+',
75                         default=['02', '59', '60', '62', '80'])
76     parser.add_argument('--container', default='raw')
77     parser.add_argument('--datalake-path', default='geo')
78     parser.add_argument('--local-output', default='data')
79     parser.add_argument('--connection-string')
80
81     args = parser.parse_args()
82     if args.connection_string:
83         os.environ['AZURE_STORAGE_CONNECTION_STRING'] = args.
connection_string
84
85     depts = args.departements if args.departements else []
86     fetch_communes(depts, args.container, args.datalake_path,
87                   args.local_output)

```

## B.2 Script de préparation des données (data\_prep.py)

```

1  """
2  Module de preparation et nettoyage des donnees territoriales
3  Fichier : analytics/lib/data_prep.py
4  """
5  import pandas as pd
6  import glob
7  import os
8
9  def prepare_tables():
10     """
11     Prepare et nettoie les donnees depuis sources multiples
12
13     Returns:
14         DataFrame : Donnees consolidees et nettoyees
15     """
16     print("[INFO] Debut preparation des tables...")

```

```
17
18 # 1. Lecture JSON depuis API Geo
19 print("[INFO] Lecture fichiers JSON communes...")
20 json_files = glob.glob('data/communes_*.json')
21
22 if not json_files:
23     print("[WARN] Aucun fichier JSON trouve")
24     return None
25
26 dfs_api = []
27 for file in json_files:
28     df = pd.read_json(file)
29     dfs_api.append(df)
30
31 communes_api = pd.concat(dfs_api, ignore_index=True)
32 print(f"[OK] {len(communes_api)} communes chargees depuis API
33 ")
34
35 # 2. Lecture CSV supplementaires (si disponibles)
36 csv_files = glob.glob('data/*.csv')
37 if csv_files:
38     print(f"[INFO] Lecture {len(csv_files)} fichiers CSV...")
39     communes_csv = pd.read_csv(csv_files[0])
40     # Fusion sur code commune
41     df_merged = pd.merge(
42         communes_api, communes_csv,
43         on='code', how='outer', suffixes=('', '_csv')
44     )
45     print(f"[OK] Fusion effectuee : {len(df_merged)} lignes")
46 else:
47     df_merged = communes_api
48
49 # 3. Nettoyage des donnees
50 print("[INFO] Nettoyage des donnees...")
51 # Suppression doublons
52 nb_before = len(df_merged)
53 df_clean = df_merged.drop_duplicates(subset='code', keep='
54 first')
55 nb_duplicates = nb_before - len(df_clean)
56 print(f"[OK] {nb_duplicates} doublons supprimes")
```

```
56  # Suppression valeurs manquantes critiques
57  nb_before = len(df_clean)
58  df_clean = df_clean.dropna(subset=['code', 'nom'])
59  nb_missing = nb_before - len(df_clean)
60  print(f"[OK] {nb_missing} lignes avec valeurs manquantes
    supprimees")
61
62  # Filtrage valeurs aberrantes
63  if 'population' in df_clean.columns:
64      df_clean = df_clean[
65          (df_clean['population'] >= 0) &
66          (df_clean['population'] < 3000000)
67      ]
68      print(f"[OK] Filtrage valeurs aberrantes population")
69
70  # 4. Normalisation
71  print("[INFO] Normalisation des formats...")
72  # Colonnes en snake_case
73  df_clean.columns = [
74      col.lower().replace(' ', '_').replace('-', '_')
75      for col in df_clean.columns
76  ]
77  # Types de donnees
78  if 'population' in df_clean.columns:
79      df_clean['population'] = pd.to_numeric(
80          df_clean['population'], errors='coerce'
81      )
82  if 'code_postal' in df_clean.columns:
83      df_clean['code_postal'] = (
84          df_clean['code_postal'].astype(str).str.zfill(5)
85      )
86
87  # 5. Export final
88  output_file = 'data/communes_final.csv'
89  df_clean.to_csv(output_file, index=False, encoding='utf-8')
90  print(f"[OK] Export final : {output_file}")
91  print(f"[OK] {len(df_clean)} communes finales")
92
93  return df_clean
94
95  if __name__ == "__main__":
```

```

96     df_final = prepare_tables()
97     if df_final is not None:
98         print("\n[SUCCESS] Preparation terminée")
99         print(f"Colonnes : {', '.join(df_final.columns[:10])}")
100        print(f"Shape : {df_final.shape}")

```

### B.3 Script d'export SQL (export\_to\_sql.py)

```

1  """
2  Script d'export des donnees preparees vers Azure SQL Database
3  Fichier : analytics/export_to_sql.py
4  """
5  import pandas as pd
6  import pyodbc
7  import os
8  from dotenv import load_dotenv
9
10 load_dotenv()
11
12 def get_sql_connection():
13     """Cree connexion vers Azure SQL Database"""
14     server = os.getenv('AZURE_SQL_SERVER', 'sqllelbrek.database.
15 windows.net')
16     database = os.getenv('AZURE_SQL_DATABASE', 'projet_data_eng')
17     username = os.getenv('AZURE_SQL_LOGIN', 'sqladmin')
18     password = os.getenv('AZURE_SQL_PASSWORD')
19
20     if not password:
21         raise ValueError("AZURE_SQL_PASSWORD non definie")
22
23     conn_str = (
24         f'DRIVER={{ODBC Driver 18 for SQL Server}};'
25         f'SERVER={server};'
26         f'DATABASE={database};'
27         f'UID={username};'
28         f'PWD={password};'
29         f'Encrypt=yes;'
30         f'TrustServerCertificate=no;'
31         f'Connection Timeout=30;'
32     )
33     return pyodbc.connect(conn_str)

```

```

34 def create_tables_if_not_exist(cursor):
35     """Cree les tables si elles n'existent pas"""
36     ddl_regions = """
37     IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='regions')
38     CREATE TABLE regions (
39         code_region VARCHAR(2) PRIMARY KEY,
40         nom_region VARCHAR(100) NOT NULL,
41         created_at DATETIME DEFAULT GETDATE()
42     );"""
43
44     ddl_departements = """
45     IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='
46     departements')
47     CREATE TABLE departements (
48         code_dept VARCHAR(3) PRIMARY KEY,
49         nom_dept VARCHAR(100) NOT NULL,
50         code_region VARCHAR(2),
51         created_at DATETIME DEFAULT GETDATE(),
52         FOREIGN KEY (code_region) REFERENCES regions(code_region)
53     );"""
54
55     ddl_communes = """
56     IF NOT EXISTS (SELECT * FROM sys.tables WHERE name='communes
57     ')
58     CREATE TABLE communes (
59         code_commune VARCHAR(5) PRIMARY KEY,
60         nom_commune VARCHAR(100) NOT NULL,
61         population INT CHECK (population >= 0),
62         surface DECIMAL(10,2),
63         code_dept VARCHAR(3),
64         created_at DATETIME DEFAULT GETDATE(),
65         FOREIGN KEY (code_dept) REFERENCES departements(code_dept
66     )
67     );"""
68
69     cursor.execute(ddl_regions)
70     cursor.execute(ddl_departements)
71     cursor.execute(ddl_communes)
72     print("[OK] Tables creees ou deja existantes")
73
74 def export_to_sql():

```

```

72     """Exporte les donnees preparees vers Azure SQL"""
73     print("[INFO] Connexion a Azure SQL Database...")
74     conn = get_sql_connection()
75     cursor = conn.cursor()
76
77     create_tables_if_not_exist(cursor)
78     conn.commit()
79
80     print("[INFO] Lecture donnees preparees...")
81     df_communes = pd.read_csv('data/communes_final.csv')
82     print(f"[INFO] Import de {len(df_communes)} communes...")
83
84     for idx, row in df_communes.iterrows():
85         try:
86             cursor.execute("""
87                 INSERT INTO communes
88                 (code_commune, nom_commune, population, code_dept
89 )
90                 VALUES (?, ?, ?, ?)
91             """, row['code'], row['nom'],
92                 row.get('population'), row.get('code_departement'
93 ))
94
95         except pyodbc.IntegrityError:
96             continue
97
98         if (idx + 1) % 1000 == 0:
99             print(f"[INFO] {idx+1} lignes traitees...")
100             conn.commit()
101
102     conn.commit()
103     cursor.execute("SELECT COUNT(*) FROM communes")
104     count = cursor.fetchone()[0]
105     print(f"[OK] {count} communes importees dans Azure SQL")
106     conn.close()
107
108 if __name__ == "__main__":
109     export_to_sql()

```

## B.4 Notebook Databricks – parquet\_etl (PySpark)

Ce notebook est exécuté via le job **parquet-etl** sur le cluster **etl-parquet** dans le workspace Databricks **dbw-elbrek-e4**. Il lit le fichier SIRENE depuis la zone **raw/** et écrit les données nettoyées dans la zone **curated/** de l'ADLS Gen2.

```

1 # =====
2 # Notebook : /Shared/parquet_etl
3 # Objectif : Nettoyer SIRENE - Hauts-de-France actifs uniquement
4 # =====
5
6 # -- CELLULE 1 : Configuration ADLS --
7 STORAGE_ACCOUNT = "adlsselbrek2025"
8 CONTAINER_RAW = "raw"
9 CONTAINER_CURATED = "curated"
10 STORAGE_KEY = "<CLE_ACCES_ADLS>"
11
12 spark.conf.set(
13     f"fs.azure.account.key.{STORAGE_ACCOUNT}.dfs.core.windows.net",
14     STORAGE_KEY
15 )
16
17 # -- CELLULE 2 : Chemins ADLS --
18 PATH_SOURCE = (f"abfss://{CONTAINER_RAW}@{STORAGE_ACCOUNT}"
19               ".dfs.core.windows.net/Parquet/"
20               "StockEtablissement_utf8.parquet")
21 PATH_OUTPUT = (f"abfss://{CONTAINER_CURATED}@{STORAGE_ACCOUNT}"
22               ".dfs.core.windows.net/etablislements_hdf_actifs/"
23               )
24
25 # -- CELLULE 3 : Lecture du fichier brut (53 colonnes, >30M
26   lignes) --
27 df_raw = spark.read.parquet(PATH_SOURCE)
28
29 # -- CELLULE 4 : Selection des 21 colonnes essentielles --
30 COLONNES = [
31     "siret", "siren", "nic", "etablissementSiege",
32     "etatAdministratifEtablissement", "dateCreationEtablissement",
33     "dateDebut", "dateDernierTraitementEtablissement",
34     "activitePrincipaleEtablissement",
35     "nomenclatureActivitePrincipaleEtablissement",

```



```

34     "activitePrincipaleRegistreMetiersEtablissement",
35     "trancheEffectifsEtablissement", "anneeEffectifsEtablissement",
36     "caractereEmployeurEtablissement", "numeroVoieEtablissement",
37     "typeVoieEtablissement", "libelleVoieEtablissement",
38     "codePostalEtablissement", "libelleCommuneEtablissement",
39     "codeCommuneEtablissement", "denominationUsuelleEtablissement",
40     "enseigne1Etablissement",
41 ]
42 df = df_raw.select(COLONNES)
43
44 # -- CELLULE 5 : Filtre etablissements actifs (etat = 'A') --
45 from pyspark.sql.functions import col, substring, lit,
46     current_date
47 from pyspark.sql.functions import to_date, create_map
48 from itertools import chain
49 df = df.filter(col("etatAdministratifEtablissement") == "A")
50
51 # -- CELLULE 6 : Filtre Hauts-de-France (depts 02/59/60/62/80) --
52 DEPTS_HDF = ["02", "59", "60", "62", "80"]
53 df = df.filter(
54     substring(col("codeCommuneEtablissement"), 1, 2).isin(
55         DEPTS_HDF)
56 )
57 # -- CELLULE 7 : Suppression des colonnes entierement vides --
58 total = df.count()
59 colonnes_vides = [c for c in df.columns
60     if df.filter(col(c).isNull()).count() == total]
61 if colonnes_vides:
62     df = df.drop(*colonnes_vides)
63
64 # -- CELLULE 8 : Enrichissement et renommage --
65 NOM_DEPT = {"02": "Aisne", "59": "Nord", "60": "Oise",
66     "62": "Pas-de-Calais", "80": "Somme"}
67 mapping = create_map([lit(x) for x in chain(*NOM_DEPT.items())])
68
69 df = (df
70     .withColumn("code_departement",

```

```

71     substring(col("codeCommuneEtablissement"), 1, 2))
72     .withColumn("nom_departement", mapping[col("code_departement"
73     )])
74     .withColumn("region", lit("Hauts-de-France"))
75     .withColumn("date_traitement", current_date())
76     .withColumn("dateCreationEtablissement",
77     to_date(col("dateCreationEtablissement"), "yyyy-MM-dd"))
78     .withColumn("dateDebut",
79     to_date(col("dateDebut"), "yyyy-MM-dd"))
80     .withColumnRenamed("etablissementSiege", "est_siege"
81     )
82     .withColumnRenamed("etatAdministratifEtablissement", "etat")
83     .withColumnRenamed("dateCreationEtablissement", "
84     date_creation")
85     .withColumnRenamed("dateDebut", "
86     date_debut_periode")
87     .withColumnRenamed("dateDernierTraitementEtablissement", "
88     date_maj")
89     .withColumnRenamed("activitePrincipaleEtablissement", "
90     code_naf")
91     .withColumnRenamed("trancheEffectifsEtablissement", "
92     tranche_effectifs")
93     .withColumnRenamed("caractereEmployeurEtablissement", "
94     est_employeur")
95     .withColumnRenamed("numeroVoieEtablissement", "num_voie")
96     .withColumnRenamed("typeVoieEtablissement", "type_voie"
97     )
98     .withColumnRenamed("libelleVoieEtablissement", "nom_voie")
99     .withColumnRenamed("codePostalEtablissement", "
code_postal")
100    .withColumnRenamed("libelleCommuneEtablissement", "commune")
101    .withColumnRenamed("codeCommuneEtablissement", "
code_commune")
102    .withColumnRenamed("denominationUsuelleEtablissement", "
nom_commercial")
103    .withColumnRenamed("enseigne1Etablissement", "enseigne")
104 )
105
106 # -- CELLULE 9 : Apercu statistiques --
107 df.groupBy("code_departement", "nom_departement").count() \
108     .orderBy("code_departement").show()

```

```

100 display(df.limit(20))
101
102 # -- CELLULE 10 : Sauvegarde zone curated, partitionnee par dept
103     --
104 df.write.mode("overwrite") \
105     .partitionBy("code_departement") \
106     .parquet(PATH_OUTPUT)
107
108 # -- CELLULE 11 : Verification --
109 files = dbutils.fs.ls(PATH_OUTPUT)
110 for f in files:
111     print(f.name)

```

## B.5 Script export Azure SQL projet\_data\_eng\_bis (sql\_to\_adls\_bis.py)

Ce script extrait les 13 tables de la base Azure SQL projet\_data\_eng\_bis hébergée sur le serveur sqllelbrek-prod2.database.windows.net et les exporte en Parquet vers la zone raw/sql-bis/ de l'ADLS Gen2 (adlsselbrek2025). Il est disponible dans analytics/sql\_to\_adls\_bis.py.

```

1  """
2  Export tables Azure SQL (instance BIS) -> ADLS Gen2 (format
3  Parquet)
4  Fichier : analytics/sql_to_adls_bis.py
5  """
6  import pandas as pd, sqlalchemy as sa, io, urllib
7  from azure.storage.blob import BlobServiceClient, ContentSettings
8
9  TABLES = [
10      "dim_commune", "bridge_commune_code_postal",
11      "stg_population", "stg_creation_entreprises",
12      "stg_creation_entrepreneurs_individuels", "stg_deces",
13      "stg_ds_filosofi", "stg_emploi_chomage", "stg_fecondite",
14      "stg_filosofi_age_tp_nivvie", "stg_logement",
15      "stg_menage", "stg_naissances",
16  ]
17
18 def create_engine(server, database, username, password, driver):
19     params = urllib.parse.quote_plus(
20         f"Driver={{driver}};Server=tcp:{server},1433;"
21         f"Database={database};Uid={username};Pwd={password};"
22         f"Encrypt=yes;TrustServerCertificate=yes;"

```

```

22     )
23     return sa.create_engine(
24         f"mssql+pyodbc:///odbc_connect={params}",
25         fast_executemany=True
26     )
27
28 def upload_parquet(df, service, container, blob_name):
29     buf = io.BytesIO()
30     df.to_parquet(buf, engine="pyarrow", index=False)
31     service.get_blob_client(container=container, blob=blob_name) \
32         .upload_blob(buf.getvalue(), overwrite=True,
33                     content_settings=ContentSettings(
34                         content_type="application/octet-
35 stream"))
36
37 def main():
38     engine = create_engine(SERVER, DATABASE, USERNAME, PASSWORD,
39                             DRIVER)
40     service = BlobServiceClient.from_connection_string(
41         ADLS_CONN_STR)
42     for table in TABLES:
43         df = pd.read_sql(f"SELECT * FROM dbo.{table}", engine)
44         blob = f"sql-bis/{table}.parquet"
45         upload_parquet(df, service, "raw", blob)
46         print(f"[OK] {table} -> raw/{blob} ({len(df)} lignes)")
47     engine.dispose()
48
49 if __name__ == "__main__":
50     main()

```

## C Tests API avec Postman

Cette annexe présente les captures d'écran des tests effectués avec Postman.

## C.1 Test endpoint /health

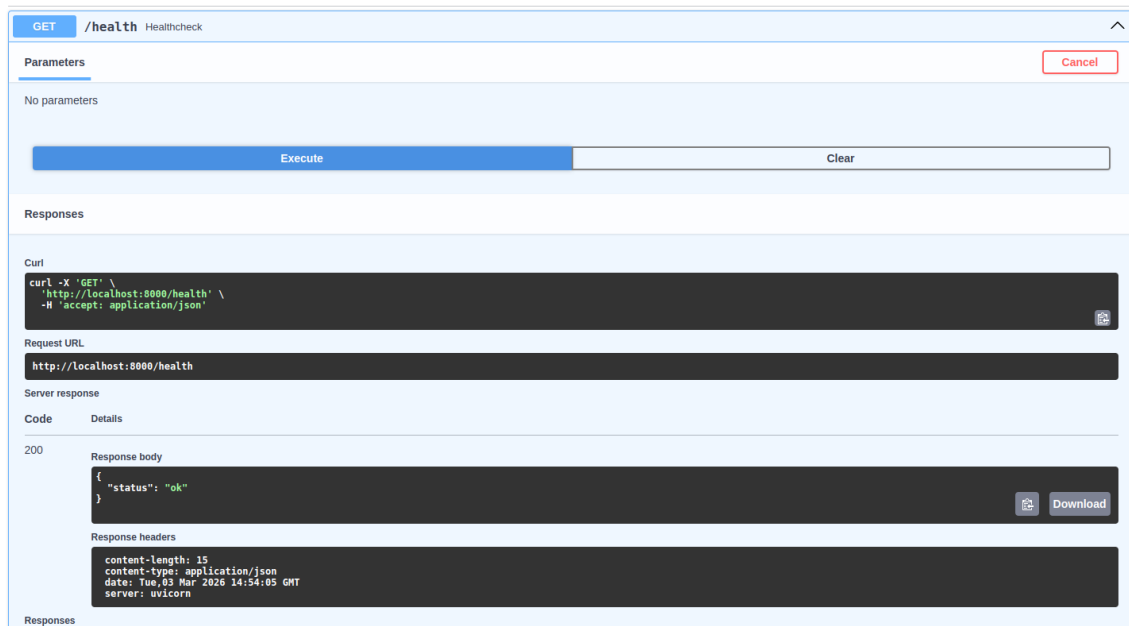


FIGURE 10 – Fenêtre Postman complète montrant : (1) la requête `GET http://localhost:8000/health`, (2) la réponse `200 OK`, (3) le corps JSON retourné (ex : `{"status": "ok", "version": "1.0"}`).

## C.2 Test endpoint /communes avec authentication

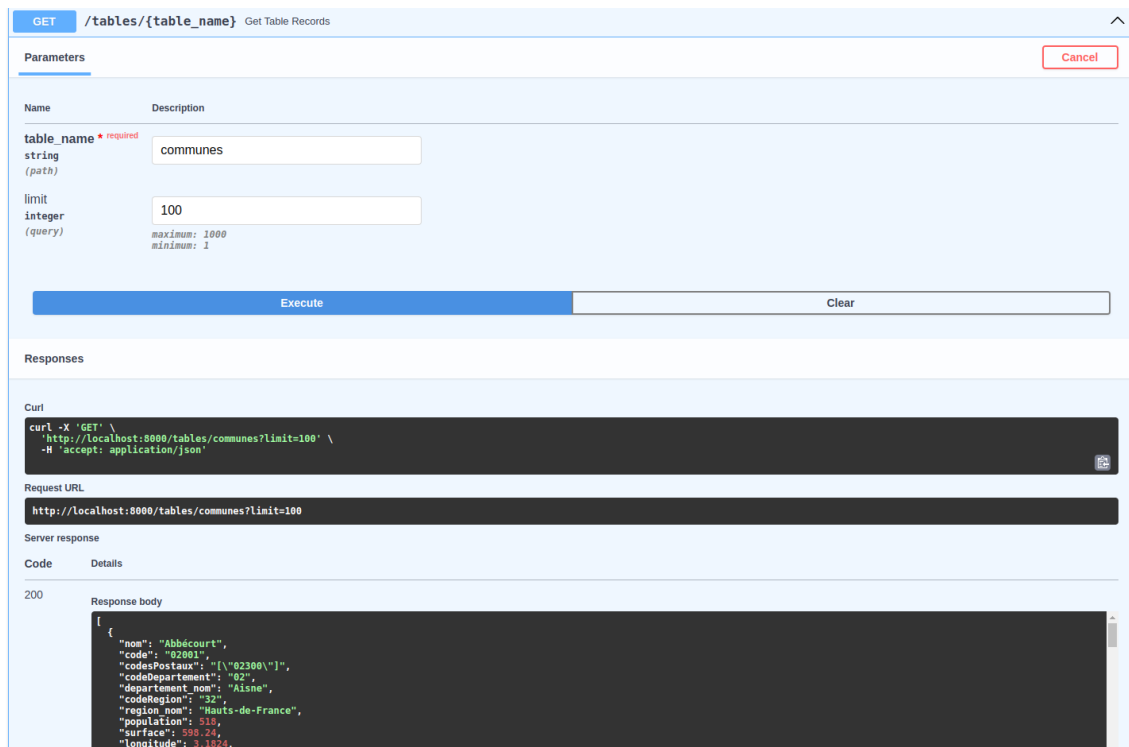


FIGURE 11 – Fenêtre Postman complète montrant : (1) la requête `GET http://localhost:8000/communes`, (2) l'onglet **Headers** avec `X-API-Key: <votre-clé>`, (3) la réponse **200 OK** avec le tableau JSON des communes.

## C.3 Test rejet sans API Key

**TODO – Capture d'écran à ajouter**

Fichier attendu : `screenshots/postman_403.png`

Ce qu'il faut capturer : Fenêtre Postman complète montrant : (1) la requête `GET http://localhost:8000/communes` sans header `X-API-Key`, (2) la réponse **403 Forbidden**, (3) le corps JSON `{"detail": "Invalid API Key"}`.

## D Références

Dépôt GitHub : <https://github.com/haelbrek/Projet-Data-ENG>

Sources de données :

- API Géo : <https://geo.api.gouv.fr>
- INSEE : <https://www.insee.fr>
- Data.gouv.fr : <https://www.data.gouv.fr>

Documentation technique :

- FastAPI : <https://fastapi.tiangolo.com>
- Azure SQL Database : <https://docs.microsoft.com/azure/sql-database>
- Pandas : <https://pandas.pydata.org>